Avant la séance 1 — Préparer votre ordinateur

Sommaire

1. Installer Java 25
   - macOS ou Linux : avec SDKMAN
   - Windows
   - Vérifier (tous les systèmes)
2. Installer IntelliJ IDEA
3. Créer et lancer votre premier programme
   - Créer un projet
   - Créer la classe `HelloWorld`
   - Lancer le programme
4. Envoyer votre capture

Ça ne marche pas ?

Facultatif : si tout fonctionne et que vous voulez prendre de l'avance

> **À faire avant la première séance**, sur l'ordinateur que vous apporterez en cours. Comptez **30 à 45 minutes**. À la fin, vous aurez installé Java 25 et IntelliJ IDEA, et lancé votre premier programme Java. **Pourquoi avant ?** Pour que la séance serve à programmer, pas à attendre des téléchargements sur le wifi de l'école.

Trois étapes : **(1)** installer Java 25, **(2)** installer IntelliJ IDEA, **(3)** créer et lancer une classe `HelloWorld`. Puis envoyez une capture d'écran (étape 4).

---

1. Installer Java 25

Choisissez la méthode qui correspond à votre système.

macOS ou Linux : avec SDKMAN

SDKMAN installe Java dans votre dossier personnel, sans droits administrateur, et permet de changer de version facilement. Dans un **terminal** :

```
curl -s "https://get.sdkman.io" | bash
```

Fermez le terminal et ouvrez-en un nouveau, puis listez les versions de Java disponibles :

```
sdk list java
```

Dans la liste, repérez la ligne **Temurin** en version **25** et copiez la valeur de la dernière colonne, **Identifier** (par exemple `25.0.5.0+8-tem` ; le numéro exact change à chaque mise à jour). Appuyez sur `q` pour quitter la liste. Installez ensuite Java et Maven :

```
sdk install java 25.0.5.0+8-tem
sdk install maven
```

Remplacez `25.0.5.0+8-tem` par l'identifiant que vous avez copié. Répondez `Y` si SDKMAN vous demande d'en faire la version par défaut.

Windows

Ouvrez **PowerShell** (menu Démarrer, tapez « PowerShell ») et lancez :

```
winget install EclipseAdoptium.Temurin.25.JDK
```

Si `winget` n'est pas disponible : téléchargez l'installeur `.msi` de **Temurin 25 (JDK)** sur adoptium.net, et pendant l'installation, **activez les options « Set JAVA_HOME » et « Add to PATH »**.

Vous n'avez pas besoin d'installer Maven sous Windows : IntelliJ IDEA en contient un.

Vérifier (tous les systèmes)

Ouvrez un nouveau terminal (ou un nouveau PowerShell) et tapez :

```
java -version
```

Vous devez voir **25** sur la première ligne, par exemple :

```
openjdk version "25.0.5" … LTS
OpenJDK Runtime Environment Temurin-25.0.5 …
```

Si vous voyez une autre version (17, 21…) ou une erreur, consultez **« Ça ne marche pas ? »** à la fin de ce document.

---

2. Installer IntelliJ IDEA

IntelliJ IDEA est l'environnement de développement que nous utiliserons en cours : il signale les erreurs pendant que vous tapez, lance les programmes et les tests, et contient un débogueur.

1. Téléchargez **IntelliJ IDEA** sur jetbrains.com/idea/download et installez-le normalement. Les fonctions utilisées dans le cours sont gratuites.
2. Facultatif : en tant qu'étudiant, vous pouvez obtenir gratuitement la licence complète sur jetbrains.com/student, avec votre adresse e-mail de l'école.
3. Lancez IntelliJ IDEA. Au premier démarrage, acceptez les réglages par défaut.

3. Créer et lancer votre premier programme

Créer un projet

Sur l'écran d'accueil, cliquez sur **New Project** et remplissez :

Champ	Valeur
Name	`seance1`
Location	un dossier de votre choix, par exemple `Documents/java`
Language	**Java**
Build system	**Maven**
JDK	**25**
Add sample code	**décoché**

Le JDK 25 n'apparaît pas dans la liste ? Ouvrez la liste *JDK* :

- si Java a été installé avec SDKMAN : *Add JDK from disk…*, puis choisissez le dossier `.sdkman/candidates/java/current` dans votre dossier personnel (sur macOS, appuyez sur ⌘⇧. pour afficher les dossiers cachés) ;
- sinon : *Download JDK…*, version **25**, fournisseur **Eclipse Temurin**.

Cliquez sur **Create**. La première ouverture peut prendre une à deux minutes : IntelliJ télécharge ce dont Maven a besoin (barre de progression en bas de la fenêtre). Attendez qu'elle se termine.

Créer la classe `HelloWorld`

Dans le panneau de gauche, dépliez `seance1` → `src` → `main` → `java`. Faites un **clic droit sur `java` → New → Java Class**, tapez `HelloWorld`, puis Entrée.

Remplacez le contenu du fichier par :

```java
// Fichier : HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, FoodFast !");
        System.out.println("Java " + System.getProperty("java.version"));
    }
}
```

Le nom de la classe (`HelloWorld`) doit être **exactement** celui du fichier (`HelloWorld.java`), majuscules comprises.

Lancer le programme

Une **flèche verte** ▶ apparaît dans la marge, à gauche de la ligne `public class HelloWorld` et de la ligne `main`. Cliquez dessus, puis sur **Run 'HelloWorld.main()'**.

La fenêtre **Run** s'ouvre en bas et affiche :

```
Hello, FoodFast !
Java 25.0.5
```

La deuxième ligne doit commencer par **Java 25**.

---

4. Envoyer votre capture

Répondez au message qui vous a transmis ce document avec **une capture d'écran** qui montre :

- la fenêtre **Run** d'IntelliJ avec `Hello, FoodFast !` et `Java 25…` ;
- **ou**, si vous êtes bloqué, le message d'erreur que vous obtenez, et à quelle étape.

Il n'y a aucune note : c'est pour savoir avant la séance qui a besoin d'aide.

Ça ne marche pas ?

Problème	Solution
`sdk: command not found`	Fermez le terminal et ouvrez-en un nouveau. Sinon : `source "$HOME/.sdkman/bin/sdkman-init.sh"`
`java -version` affiche 17, 21 ou une autre version	macOS/Linux : `sdk default java <identifiant 25>`. Windows : désinstallez l'ancienne version de Java (Paramètres → Applications), puis ouvrez un nouveau PowerShell
`java` n'est pas reconnu (Windows)	Ouvrez un **nouveau** PowerShell. Si l'erreur persiste, réinstallez avec le `.msi` en cochant « Add to PATH »
La flèche verte ▶ n'apparaît pas	Vérifiez que `HelloWorld.java` est bien dans `src/main/java`, et attendez la fin de la barre de progression en bas d'IntelliJ
`Java 21` (ou autre) s'affiche dans la fenêtre Run	Le projet n'utilise pas le bon JDK : menu **File** → **Project Structure** → **Project** → **SDK**, choisissez 25
`class HelloWorld is public, should be declared in a file named HelloWorld.java`	Le nom du fichier et celui de la classe diffèrent : renommez l'un ou l'autre
Autre chose	Faites une capture du message et envoyez-la : on règle ça avant la séance

---

Facultatif : si tout fonctionne et que vous voulez prendre de l'avance

Le TP de ce module consiste à écrire **votre propre serveur compatible Redis**. Pour comparer votre serveur au vrai, nous utiliserons **Docker**. Si vous le souhaitez, installez Docker Desktop, puis vérifiez dans un terminal :

```
docker run --rm hello-world
```

Le message `Hello from Docker!` confirme que Docker fonctionne. Ce n'est pas obligatoire avant la séance 1.